

Learning on SOMs

Given a SOM with weights $w_{i,j}$

i - component

j - node

and an input we learn in the following way.

0. Initialize nodes with random weights.

1. Given input x , find nearest node w_k .

$$D(x, w_j) = \sum_i (x_i - w_{i,j})^2$$

$$k = \underset{j}{\operatorname{argmin}} (D(x, w_j))$$

2. Update weights.

$$w_{i,j} \leftarrow w_{i,j} + \alpha \cdot h(j,k) (x_i - w_{i,j})$$

$h(j,k)$ "neighbourhood function"

- describes how near w_j is to w_k

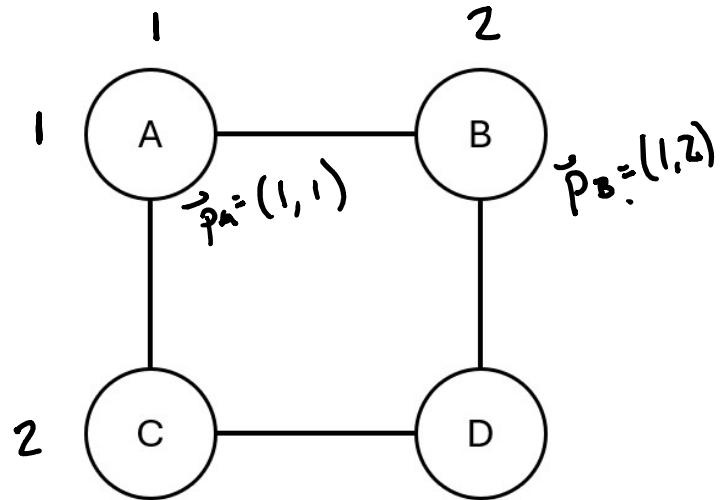
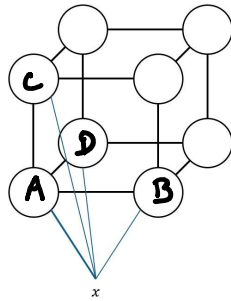
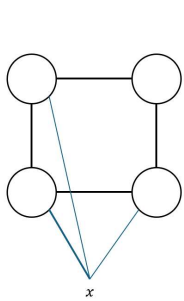
α - learning rate

3. Repeat over all inputs x . Until all inputs are mapped to the same node for several iterations.

1.1. Neighbourhood Function

Adjacency Neighbourhood Function

$$h(j, k) = \begin{cases} 1 & \text{if } j=k \text{ or } j \text{ and } k \text{ adjacent} \\ 0 & \text{otherwise} \end{cases}$$



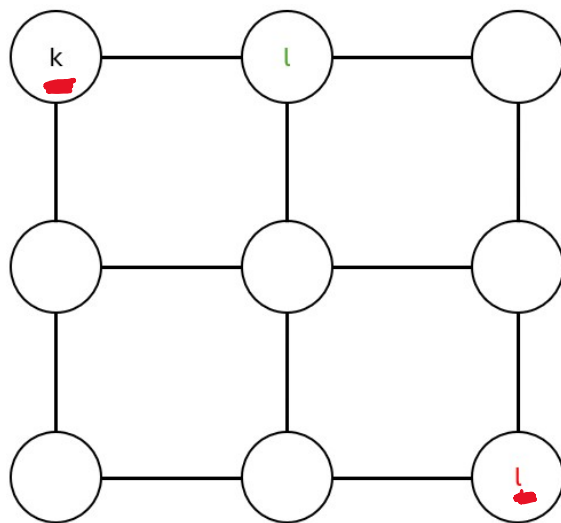
Gaussian Neighbourhood Function

$$h(j, k) = e^{-\beta |\vec{p}_j - \vec{p}_k|^2}$$

$\vec{p}_i$ - position coordinates of node i
in graph

β - hyperparameter controlling
width of Gaussian

Measuring Performance



Quantization Error

$$QE = \sqrt{\frac{1}{n} \sum_d \sum_i (x_i - w_{i,k})^2}$$

k - the closest node

d - sums over all inputs

n - number of inputs

Topological Error

Given input x .

k - the nearest node

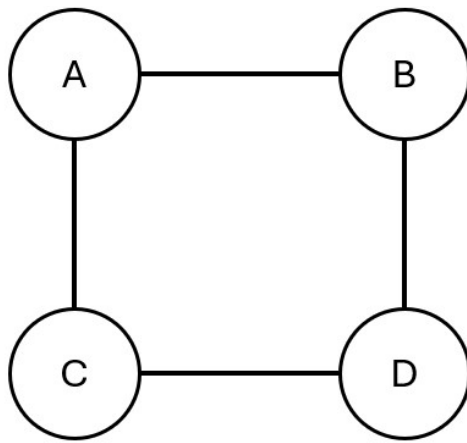
l - the second nearest node

$$TE = \frac{1}{n} \sum_d (1 - h(k, l))$$

Example



$$w_n = [0, 0, 1]$$



$$w_A = [0, 0, 1]$$

$$w_B = [-1, 0, 0]$$

$$w_C = [1, 1, -1]$$

$$w_D = [0, -1, 1]$$

Use adjacency neighbourhood function.
 $\alpha = 0.1$

$$x = [1, -1, 1]$$

$$1. \quad D(x, w_A) = 2$$

$$D(x, w_B) = 6$$

$$D(x, w_C) = 8$$

$$D(x, w_D) = 1$$

The closest node to x is D.

$$2. \quad w_A \leftarrow [0, 0, 1] + 0.1 \cdot 0 \left([1, -1, 1] - [0, 0, 1] \right) \\ \leftarrow [0, 0, 1]$$

$$w_B \leftarrow [-1, 0, 0] + 0.1 \cdot 1 \left([1, -1, 1] - [-1, 0, 0] \right) \\ \leftarrow [-0.8, -0.1, 0.1]$$

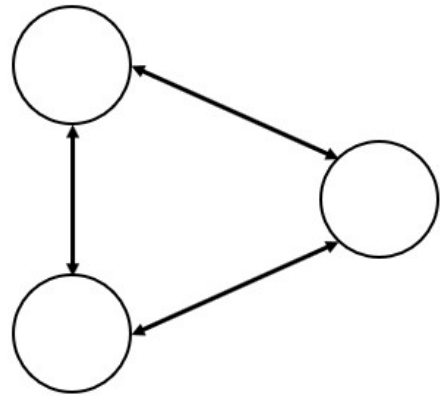
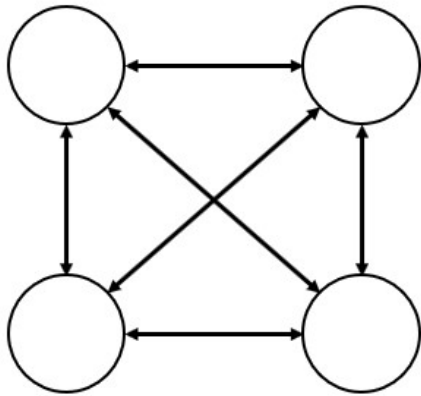
$$w_C \leftarrow [1, 1, -1] + 0.1 \cdot 1 \cdot \left([1, -1, 1] - [1, 1, -1] \right) \\ \leftarrow [1, 0.8, -0.8]$$

$$w_D \leftarrow [0, -1, 1] + 0.1 \cdot 1 \cdot \left([1, -1, 1] - [0, -1, 1] \right) \\ \leftarrow [0.1, -1, 1]$$

$$\begin{aligned} &\leftarrow [1, 0.8, -0.8] \\ w_D &\leftarrow [0, -1, 1] + 0.1 \cdot 1 \cdot ([1, -1, 1] - [0, -1, 1]) \\ &\leftarrow [0.1, -1, 1] \end{aligned}$$

Repeat...

Hopfield Networks



Every neuron receives signal from every other neuron.

Every neuron outputs signal to every other neuron.

Hopfield Network

A neural network where every neuron is connected to every other neuron subject to

Define $w_{i,j}$ as the weight of connection
 - to neuron j .

Define $w_{i,j}$ as the weight from neuron i to neuron j .

1. $w_{i,j} = w_{j,i}$

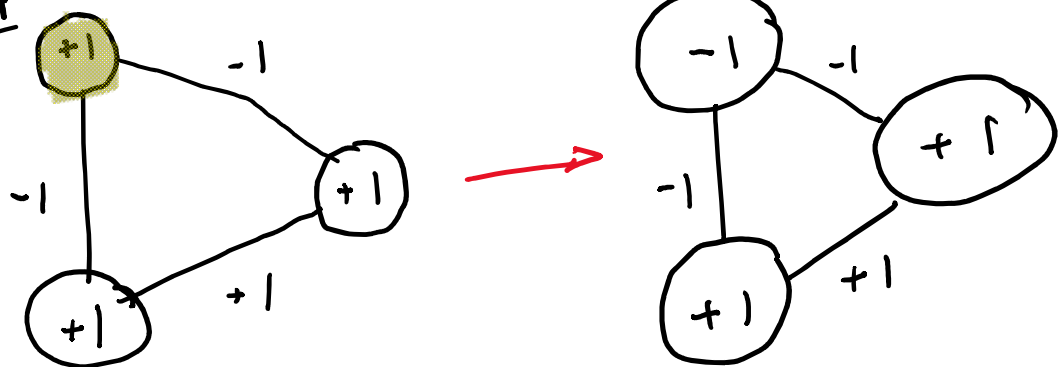
2. Aggregation: $f(\vec{x}) = \sum_i w_i x_i$

3. Activation:
$$a(x) = \begin{cases} -1 & \text{if } x < 0 \\ \text{unchanged} & \text{if } x = 0 \\ 1 & \text{if } x > 0 \end{cases}$$

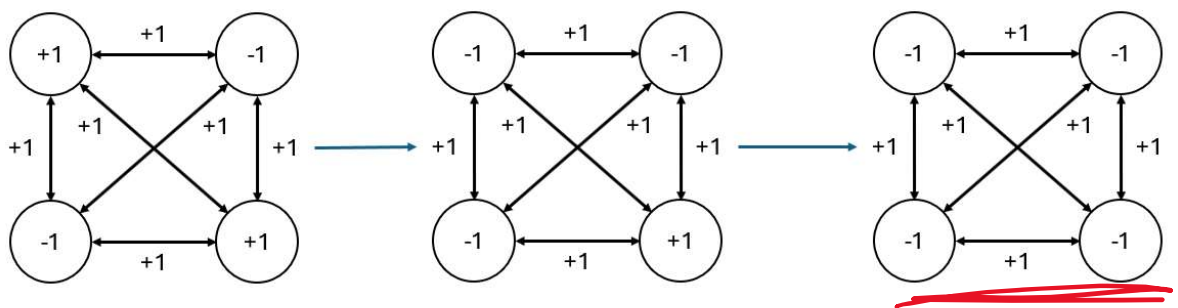
Asynchronous updates

Update activations of neurons one at a time.

Example



Example



Consider x_i^B - output i^{th} neuron before update

Consider x_i^B - output i^{th} neuron before update
 x_i^F - output i^{th} neuron after update

$$x_i^F \sum_{j \neq i} w_{j,i} x_j - x_i^B \sum_{j \neq i} w_{j,i} x_j$$

Case 1 $x_i^F = x_i^B$

Expression is zero

Case 2 $x_i^F = -x_i^B$

Expression is positive

Any time a neuron's activation is updated
 $x_i \sum_{j \neq i} w_{j,i} x_j$ increases

This has an upper bound.

The Hopfield network will converge to steady state.

Stationary point: A set of activations x_i
such that for all i :

$$x_i = \alpha \left(\sum_{j \neq i} w_{j,i} x_j \right)$$

Claim: For input $\vec{x}$, $w_{j,i} = x_j x_i$ then $\vec{x}$ is a stationary point.

Proof

$$\begin{aligned}
 & a\left(\sum_{j \neq i} w_{j,i} x_j\right) \\
 &= a\left(\sum_{j \neq i} x_j x_i x_j\right) \\
 &= a\left(\sum_{j \neq i} x_i\right) \\
 &= a(x_i) \\
 &= x_i
 \end{aligned}$$

Thus, the network converges to $\vec{x}$ if $w_{j,i} = x_j x_i$.

Analogously, if we want our network to converge to one of a set of patterns $\vec{x}^p$ then

$$w_{j,i} = \sum_p x_j^p x_i^p$$

Example

→ ... Hopfield network

Example

3 neuron Hopfield network
Use 3 patterns to compute weights.

$$\vec{x}^{p1} = [1, 1, 1]$$

$$\vec{x}^{p2} = [-1, 1, 1]$$

$$\vec{x}^{p3} = [1, 1, -1]$$

How do we compute weights?

$$w_{1,2} = \sum_p x_1 x_2 = 1$$

$$w_{1,3} = \sum_p x_1 x_3 = -1$$

$$w_{2,3} = \sum_p x_2 x_3 = 1$$